

CMPT 280

Topic 7: Abstract Data Types (ADTs)

Mark G. Eramian

University of Saskatchewan

References

- Textbook, Chapter 7

Reading Referesh: Quiz

1. What is a data structure?
2. What is the difference between a data type and a data structure?
3. What is an abstract data type?
4. Why do we use ADTs?
5. What is the difference between specification and implementation?

A List ADT

Name: List(G)

Sets:

L : set of lists containing items from G

G : set of items that can be in the list

B : {true, false}

Signatures:

newList(G) : $\rightarrow L$

L .isEmpty: $\rightarrow B$

L .insertFirst(g): $G \rightarrow L$

L .firstItem: $\nrightarrow G$

L .deleteFirst: $\nrightarrow L$

Preconditions: For all $l \in L$, $g \in G$

newList(G): none

l .isEmpty: none

l .insertFirst(g): none

l .firstItem: l is not empty

l .deleteFirst: l is not empty

Semantics: For $l \in L$, $g \in G$

newList(G): construct new empty list to hold elements from G .

l .isEmpty: return true if l is empty, false otherwise

l .insertFirst(g): g is added to l at the beginning.

l .firstItem: returns the first item in l .

l .deleteFirst: removes the first item in l .

Note: the symbol $\nrightarrow$ denotes a partial function.

A Student ADT

Name: Student

Sets:

S : set of all students

N : set of all names

K : set of all student numbers

Signatures:

$\text{newStudent}(n, k) : N \times K \rightarrow S$

$S.\text{getName} : \rightarrow N$

$S.\text{getNumber} : \rightarrow K$

$S.\text{setName}(n) : N \rightarrow S$

$S.\text{setNumber}(k) : K \rightarrow S$

Preconditions: For all $s \in S, n \in N, k \in K$

newStudent : none

$s.\text{getName}$: none

$s.\text{getNumber}$: none

$s.\text{setName}(n)$: none

$s.\text{setNumber}(k)$: none

Semantics: For $s \in S, n \in N, k \in K$

$\text{newStudent}(n, k)$: construct new student with name $n \in N$ and number $k \in K$

$s.\text{getName}$: return name of s

$s.\text{getNumber}$: return student number of s

$s.\text{setName}(n)$: change name of s to n .

$s.\text{setNumber}(k)$: change student number of s to k .

Exercise 1: ADT for a Queue with Bounded Size

Fill in the blanks

Name: Queue(G)

Sets:

Q : set of queues containing items from G

G : set of items that can be in the queue

B : {true, false}

$\mathbb{N}_0$: set of non-negative integers

Signatures:

newQueue(G)(n) :

Q .isEmpty:

Q .isFull:

Q .add(g):

Q .remove:

Preconditions: For all $q \in Q$, $g \in G$, $n \in \mathbb{N}_0$

newQueue(G)(n):

q .isEmpty:

q .isFull:

q .add(g):

q .remove:

Semantics: For $q \in Q$, $g \in G$, $n \in \mathbb{N}_0$

newQueue(G)(n) : create a queue of items from G with capacity n

q .isEmpty: returns true if q is empty, false otherwise

q .isFull: return true if q is full, false otherwise

q .add(g): enqueues g at the back of the queue

q .remove: removes then returns the item at the front of the queue

Exercise 2

- Write an ADT for a stack with bounded size.

Specification vs. Implementation

- Specifications do not say anything about the implementation, only the interface.
- We did not specify the specific pieces of data the ADT will need.
 - Student ADT: no specific types for name and student number.
 - List ADT: no head or tail fields for the list.
 - Queue ADT: no fields to keep track of the capacity of the queue or the number of elements it contains.
- We did not specify any underlying data structures:
 - List could be arrayed or linked.
 - Queue could use array, list.
- All of these decisions are saved for the implementation.

Implementing ADTs in Java

- Choose data types for sets.
- Type parameters remain type parameters for the class.
- Signatures become class methods.
- Preconditions become `@precond` entries for Javadoc (which the method's code should verify before proceeding) and also manifest as specific regression tests to make sure the code that verifies the preconditions operates correctly.
- Semantics become algorithms.

List Specification $\rightarrow$ Implementation

Defining the class and methods

Signatures:

$\text{newList}\langle G \rangle : \rightarrow L$

$L.\text{isEmpty} : \rightarrow B$

$L.\text{insertFirst}(g) : G \rightarrow L$

$L.\text{firstItem} : \nrightarrow G$

$L.\text{deleteFirst} : \nrightarrow L$

```
1 public class List<G> {  
2     public List() {}  
3     public boolean isEmpty() {}  
4     public void insertList(G e) {}  
5     public G firstItem() {}  
6     public void deleteFirst() {}  
7 }
```

Observe that if the signature contains $\rightarrow L$ and just modifies the state of a list rather than making a new one, then the return type is void.

Intermezzo: ADT Specification is Language Independent!

Signatures:

$\text{newList}\langle G \rangle : \rightarrow L$

$L.\text{isEmpty} : \rightarrow B$

$L.\text{insertFirst}(g) : G \rightarrow L$

$L.\text{firstItem} : \nrightarrow G$

$L.\text{deleteFirst} : \nrightarrow L$

```
1 class List:
2     def __init__(self):
3         ...
4     def isEmpty(self):
5         ...
6     def insertList(self, e):
7         ...
8     def firstItem(self):
9         ...
10    def deleteFirst(self):
11        ...
```

Python

```
1 template<typename G>
2 class List {
3 public:
4     List() {}
5     bool isEmpty() {}
6     void insertList(G e) {}
7     G firstItem() {}
8     void deleteFirst() {}
9 };
```

C++

```
1 typedef ... G;
2 typedef struct { ... } List;
3
4 List *createList();
5 int isEmpty(List *l);
6 void insertList(List *l, G e);
7 G firstItem(List *l);
8 void deleteFirst(List *l);
```

C

List Specification $\rightarrow$ Implementation

Adding the Preconditions

Preconditions: For all $l \in L$, $g \in G$

$\text{newList}\langle G \rangle$: none

$l.\text{isEmpty}$: none

$l.\text{insertFirst}(g)$: none

$l.\text{firstItem}$: l is not empty

$l.\text{deleteFirst}$: l is not empty

```
1 public class List<G> {
2     public newList() {}
3     public void isEmpty() {}
4     public void insertList(G e) {}
5     public G firstItem() {}
6     public void deleteFirst() {}
7 }
```

```
1 public class List<G> {
2
3     public newList() {}
4
5     public void isEmpty() {}
6
7     public void insertList(G e) {}
8
9     /**
10      * @precond The list is not empty.
11      */
12     public G firstItem() {}
13
14     /**
15      * @precond The list is not empty.
16      */
17     public void deleteFirst() {}
18 }
```

List Specification → Implementation

Checking the Preconditions.

Methods that have a precondition are written to throw an exception if the precondition isn't true, for example:

```
1  /**
2   * Get the item at the front of the list
3   * @precond The list is not empty.
4   * @throws IllegalStateException if the list is empty.
5   */
6
7  public G firstItem() throws ContainerEmpty280Exception {
8      // verify the precondition
9      if( this.isEmpty() )
10         throw new ContainerEmpty280Exception();
11
12     // Rest of implementation...
13 }
```

List Specification → Implementation

Adding the Semantics

```
1 public class List<G> {
2     /* Create a new list */
3     public newList() {}
4
5     /**
6      * Test whether the list is empty.
7      * @return true if the list is empty, false otherwise.
8      */
9     public void isEmpty() {}
10
11     /**
12      * Add an item to the beginning of the list.
13      * @param e the item to be added to the list.
14      */
15     public void insertList(G e) {}
16
17     /**
18      * Get the item at the front of the list
19      * @precond The list is not empty.
20      * @return the item at the beginning of the list.
21      */
22     public G firstItem() {}
23
24     /**
25      * Delete the first element of the list.
26      * @precond The list is not empty.
27      */
28     public void deleteFirst() {}
29 }
```

List Specification → Implementation

Add the data structures and method implementations.

```
1 public class List<G> {
2
3     // Let's use a linked list
4     // Of course, ListNode is an implementation of a ListNode ADT which we would have to specify...
5     protected ListNode<G> head;
6
7     /**
8      * Create a new list
9      */
10    public List() { head = null; }
11
12    ...
13 }
```

```
1 public class List<G> {
2     // Or maybe it's an array...
3     protected G[] listItems;
4
5
6     /**
7      * Create a new list
8      */
9     public List() { listItems = (G[]) new Object[100]; }
10
11    ...
12 }
```

Summary

- ADT specification describes a type independent of the implementation language.
- Uses set and function notation to achieve this independence.
- Set definitions, operations, preconditions and semantics are given.
- ADT can be implemented in any language.

Next Class

- Next class reading: Chapter 8: Trees.